baswe.Ai Engineer Accelerator™

Build These Six Projects

A step by step build guide for the six portfolio projects that prove you can do AI engineering work

RAG Knowledge Graph RAG for Enterprise Data

AGENTS Multi-Agent Research Assistant

LLMOPS Self-Healing LLM Gateway

EVALS LLM-as-Judge with Human Calibration

FINE-TUNING Model Distillation Pipeline

SAFETY Automated Red-Team Harness

One skill · one project · one proof

baswe.ai

Before you start

Each of these six projects maps to one skill an AI engineering interview will test. Built together they cover the full surface: retrieval, agents, operations, evaluation, model economics, and safety. Built well, any one of them carries a whole interview loop on its own.

Read this page first, because it decides whether the next forty pages are worth your time.

Sequence them, do not sprint them

Build in the order they appear. Project 1 gives you a system worth evaluating. Project 4 evaluates it. Project 6 attacks it. Project 5 makes it cheaper. That dependency chain is deliberate, and a portfolio where the projects reference each other reads as an engineer with a system rather than a student with a checklist.

If you only have time for two, build Project 1 and Project 3. Retrieval plus infrastructure is the most common shape of a real AI engineering job.

What makes a project count

- A number. Every one of these projects produces a measured result: an accuracy delta, an availability figure, an agreement score, a break even volume. A project with no number is a tutorial you followed.
- A README that opens with the result. Benchmark table or demo video first, architecture second, setup instructions last. Nobody scrolls.
- A ninety second demo video. Most reviewers will never run your code. They will watch the video, read the top of the README, and decide.
- Honest failure notes. A section on what did not work and why is the single strongest credibility signal in a portfolio repository.
- Real data. Public filings, your own documents, open datasets. Toy data produces toy results and interviewers can tell instantly.

How each project is laid out

Why it signals. What this project proves that a tutorial does not, and the interview question it answers for you.

Five phases. The build, in order, with the specific decisions and the places people get stuck.

Done when. The finish line, so you stop building and start shipping.

Resume line and mistakes. How to describe it, and the four ways these projects usually go wrong.

Estimates assume ten to twelve hours a week. Halve them if this is full time work, and do not be discouraged if a phase takes twice as long as the estimate. Phase 1 always does.

PROJECT 1 · RAG

Knowledge Graph RAG for Enterprise Data

Python · Neo4j · LangChain · Claude API · pgvector · FastAPI

Estimated build time: 3 to 4 weeks part time

Why this project signals

Almost every candidate has built vector RAG. Very few have built retrieval that answers questions requiring two or three hops across related entities. This project proves you understand where embeddings fail and what to do about it, which is exactly the follow up interviewers use to separate people who read about RAG from people who have debugged it.

PHASE 1

Extract entities and relationships into Neo4j

- Choose a corpus that actually has relationships. SEC filings, an internal wiki, research papers with citations, or product docs with dependency chains all work. A folder of unrelated blog posts has nothing to extract and the project dies here.
- Constrain the ontology before you write any code. Define five to ten entity types and eight to fifteen relationship types. An open ended extract all entities prompt produces a graph nobody can query.
- Chunk the documents, then run an extraction pass with Claude returning a strict schema: entity type, canonical name, relationship type, source chunk id, and confidence. Validate every response against the schema and retry the failures.
- Entity resolution is the hard part and the part that gets skipped. Acme Corp, Acme Corporation, and ACME must collapse into one node. Normalize, compare with embedding similarity above a tuned threshold, and store an alias list on the node.
- Write with MERGE rather than CREATE so re running ingestion is idempotent. Every edge carries source chunk id so you can cite the sentence that justified it.
- Budget the extraction cost per document before you run the full corpus, and cache results by document hash. This is where people accidentally spend two hundred dollars in an afternoon.

PHASE 2

Build the vector index alongside the graph

- Embed the same chunks into pgvector with metadata: document id, section path, date, and the entity ids that chunk mentions.
- Both stores share the same chunk ids. That shared key is the whole trick. It lets you cross from a graph path back to the original text, and from a retrieved passage into the graph neighborhood around it.
- Index with HNSW, tune ef_search, and measure recall at k on a small labeled set before moving on. Do not build the router on top of retrieval you have not measured.

PHASE 3

Route questions to the right retrieval path

- Write a cheap router: a small model call with a few shot prompt returning an enum, plus a low confidence fallback that runs both paths and merges.
- Route to the graph for connection questions, multi hop chains, comparisons across entities, and aggregations over relationships. Route to vectors for definitions, policy lookups, and single fact questions.
- On the graph path, extract entities from the question, resolve them to node ids, then run a parameterized Cypher query. Never let the model emit raw Cypher against your database. Keep a template library keyed by query type and let the model fill parameters only. This is a security control and it also makes results reproducible.
- Log every routing decision with the question and the outcome. You need this data for Phase 5 and you will not be able to reconstruct it later.

PHASE 4

Merge both sources into one grounded answer

- Graph traversal returns paths and vector search returns passages. Convert paths into readable statements before they reach the prompt, because raw triples generate awkward text.
- Deduplicate across the two sets, then assemble context with explicit labels separating graph derived facts from retrieved passages.
- Require a citation per claim, then validate that every citation resolves to a chunk id that was actually retrieved. Reject and regenerate when it does not. This costs nothing and eliminates invented sources.

PHASE 5

Benchmark against plain vector RAG and publish the delta

- Build a question set of fifty to a hundred items, stratified by difficulty: single hop, two hop, three hop, aggregation, and out of scope questions the system should refuse.
- Run both systems on the same set and report accuracy broken out by hop count. The expected shape is parity on single hop questions and a widening gap as hops increase. That curve is the story.
- Report latency and cost per query for both, plus the one time ingestion cost of the graph. Being honest that graph RAG is slower and more expensive to build is what makes the accuracy claim credible.
- This benchmark table is the portfolio artifact. Put it at the top of the README, above the architecture diagram.

Done when

A FastAPI endpoint answers questions with validated citations, and the README opens with a benchmark table comparing your system to a vector only baseline by hop count.

> **Resume line:** Built a hybrid knowledge graph and vector RAG system over N documents; raised

> multi hop question accuracy from X to Y percent against a vector baseline at Z ms p95.

Where this usually goes wrong

- An open ended ontology, which produces a graph you cannot write queries against.
- Skipping entity resolution, so the same company exists as four disconnected nodes.
- Letting the model write raw Cypher, which is both a security hole and unreproducible.
- Choosing a corpus with no real relationships, which makes the graph pointless.

Stretch goals: Incremental graph updates instead of full reingestion, community detection for corpus level summaries, and a graph visualization in the response UI.

PROJECT 2 · AGENTS

Multi-Agent Research Assistant

Python · LangGraph · Claude API · Tavily · Redis · FastAPI

Estimated build time: 3 to 4 weeks part time

Why this project signals

Interviewers have seen hundreds of agent demos and almost none that survive a restart, cap their own spend, or explain what they did. The differentiator here is not the agents. It is durable state, budgets, and step level tracing, which are the three things a production agent has and a tutorial agent does not.

PHASE 1

Build three specialists

- Planner takes a research question and returns a structured plan of sub questions. Researcher executes one sub question with search and returns findings. Writer turns findings into a report with citations.
- A specialist is a prompt plus a tool set plus an output schema. It is not a personality. Skip the you are a world class researcher framing and define Pydantic models for every hand off instead.
- Test each agent alone before wiring them together. If the planner produces vague sub questions in isolation, no orchestration will rescue it.

PHASE 2

Add a supervisor that routes and reviews

- Model the system as a LangGraph state graph. Nodes are agents, edges are supervisor routing decisions, and the shared state object is the single source of truth.
- After each researcher pass the supervisor checks the output against the plan: is this sub question answered, are sources missing, is another pass justified.
- Enforce budgets in the graph rather than in prompts: maximum sub questions, maximum searches per sub question, maximum total tokens, and a wall clock ceiling. A prompt asking the model to be efficient is not a budget.
- Allow the reviewer to send work back exactly once. Unlimited revision loops are how agent runs quietly cost twelve dollars.

PHASE 3

Give the researcher live search with source tracking

- Every finding is a record: claim, source URL, snippet, and retrieval timestamp. Findings without provenance are unusable in the writer step.

- Fetch and extract the page rather than trusting search snippets, which are truncated and often misleading.
- Deduplicate by URL and cap results per domain so a single site cannot dominate a report.
- Handle each failure path explicitly: no results, rate limited, paywalled, timed out. Each returns a structured status the supervisor can act on rather than an exception that kills the run.

PHASE 4

Persist run state in Redis

- Write graph state after every node: run id, plan, completed sub questions, findings so far, and cumulative token spend.
- Now a crashed or redeployed process resumes instead of replaying, which is the difference between a demo and a service.
- The same store gives you idempotency, since re entering a completed node returns the cached result, and it powers a status endpoint so callers can see progress.

PHASE 5

Ship a report endpoint with citations and a trace

- POST /research returns a run id immediately. GET /research/{id} returns status and then the finished report. Stream progress events so the interface is not a four minute spinner.
- Every claim in the report carries a citation, and each citation is validated against a real finding record before the report is returned.
- Expose GET /research/{id}/trace returning the full step log: which agent ran, what it received, what it produced, which tools it called, tokens, and latency per step.
- That trace endpoint is what makes this project interview proof. When they ask how you would debug an agent that returned a bad answer, you open the trace instead of describing one.

Done when

A research question returns a fully cited report in under five minutes, a killed process resumes from where it stopped, and every run has a viewable step by step trace.

> **Resume line:** Built a multi agent research system with supervisor and worker topology, durable run state in Redis, per run cost caps, and step level tracing for debugging.

Where this usually goes wrong

- Agents defined by personality prompts instead of typed schemas, which makes hand offs unreliable.
- No budget enforcement, so a single run silently costs more than the whole project.
- Treating search failures as exceptions rather than states the supervisor can route around.
- No trace, which means you cannot answer the one debugging question every interviewer asks.